

SIMATS ENGINEERING

ASSESSMENT TOOL 3

COURSE NAME: OBJECT ORIENTED ANALYSIS AND DESIGN

COURSE CODE: CSA1107

COURSE OUTCOME COVERED

CO1: Apply object-oriented concepts and software development life cycle models to design structured and modular software systems. (BL2, BL3)

Assessment Tool Weightage: 30%

Dr Madhan V

QUESTIONS: 6

Total Marks: 30

Annexure A

SHORT ANSWER TEST

Code of Conduct:

I ____M Nishanth (192511154) ____ (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: M Nishanth

Annexure A – Short Answer Test

1. Smart Hotel Management System – object-oriented concepts and relationships (5 Marks)

(a) Object-Oriented Concepts

- **Class:** A blueprint or template that defines the attributes (data) and methods (behaviour) common to a group of similar entities. Example: the class Room has attributes roomNo, type and price, and methods bookRoom() and cancelBooking().
- **Object:** A real instance of a class with its own state. Example: Room 305 (Deluxe, ₹4,500) is an object of the class Room; guest “Arun” is an object of the class Guest.
- **Inheritance:** A mechanism by which a child class acquires the properties and methods of a parent class. Example: DeluxeRoom and SuiteRoom inherit from Room and add only their own features such as a private pool or lounge.
- **Encapsulation:** Binding data and the methods that operate on it into a single unit and restricting direct access to the data. Example: the Payment class keeps cardNumber private and exposes it only through controlled methods such as processPayment().
- **Dynamic Binding:** Resolution of a method call at run time rather than compile time (run-time polymorphism). Example: calculateBill() called through a Room reference executes the version of DeluxeRoom or SuiteRoom, depending on the actual object.

(b) Object Relationships

- **Association:** A general “uses/knows” relationship between independent objects that have their own lifetimes. Example: a Guest is associated with a Reservation (one guest can make many reservations).
- **Dependency:** A weak, temporary relationship in which one class uses another only for the duration of a method call; a change in the supplier may affect the client. Example: the BookingService depends on PaymentGateway to confirm an online payment.
- **Composition:** A strong whole–part relationship in which the part cannot exist without the whole; if the whole is destroyed, the parts are destroyed too. Example: a Reservation is composed of BillingDetails and RoomCharges, which have no meaning without the reservation.

(c) Improvement in Maintainability and Reusability

Classes and encapsulation localise changes: a new payment method (e.g., UPI) is added inside the Payment class without altering booking or cancellation modules, which makes the system easy to maintain and debug. Inheritance lets new room categories reuse existing Room code, and dynamic binding allows new subclasses to be plugged in without changing the calling code. Association, dependency and composition clearly define how modules interact, reduce coupling and keep the design organised. Together, these concepts reduce duplication, speed up development and let well-tested components be reused in other hospitality systems.

2. Smart Education Management System – design principles and SOLID (5 Marks)

(a) Object-Oriented Design Principles

- **Abstraction:** Showing only the essential features of an entity while hiding unnecessary details. Example: a Student is represented only by roll number, name, courses and marks, not by every personal detail.
- **Encapsulation:** Wrapping data and methods in a class and controlling access with access modifiers. Example: the Grade class bundles marks and calculateGPA() so that other classes cannot alter the marks directly.
- **Information Hiding:** Concealing the internal implementation so that other modules depend only on a stable public interface. Example: how GPA is calculated stays hidden inside ResultService; other modules simply call getGPA(), so the formula can change without affecting them.

(b) SOLID Principles

- **Single Responsibility Principle (SRP):** A class should have only one reason to change. Example: StudentRecord handles student data, FeeManager handles fee calculation and ReportGenerator creates reports, instead of one class doing everything.
- **Dependency Inversion Principle (DIP):** High-level modules should depend on abstractions, not on concrete low-level modules. Example: NotificationService depends on an interface Messenger; EmailSender or SMSSender implements it, so the delivery method can be changed without modifying the service.

(c) Improvement in Scalability and Maintainability

Abstraction and information hiding reduce complexity and let teams modify internals without breaking other modules. SRP keeps classes small and focused, so a change to fee rules does not disturb attendance or exam modules, and testing becomes simpler. DIP decouples modules

through interfaces, so new features such as online exams, new payment gateways or additional notification channels can be added with minimal change. This makes the system scalable as the number of students, courses and services grows, and keeps long-term maintenance cost low.

3. Smart Pharmacy Management System – SDLC phases (5 Marks)

(a) & (b) Phases of SDLC and Their Significance

Phase	Explanation (Pharmacy System)	Significance
Feasibility Study	Evaluates whether the project is technically, economically, operationally and legally viable, e.g., cost of the software, availability of barcode scanners, and compliance with drug regulations.	Prevents investment in an unworkable project; supports go/no-go decisions and early risk identification.
Requirement Analysis	Gathers and documents needs from pharmacists, doctors and managers: stock tracking, expiry alerts, prescription handling, billing, supplier orders. Output: SRS document.	Forms the foundation of the project; clear requirements avoid rework, scope creep and misunderstanding.
System Design	Converts requirements into architecture, database design, UML diagrams, user-interface layouts and module design (inventory, billing, prescription).	Provides a blueprint for developers; ensures modularity, security and performance before coding starts.
Implementation	Developers write the code for each module according to the design, following coding standards, and integrate the modules.	Turns the design into a working product; quality of code affects reliability and maintainability.
Testing	Unit, integration, system and acceptance testing verify features such as correct dosage records, billing accuracy and expiry alerts.	Detects defects before release; ensures safety, correctness and user satisfaction, which is critical in healthcare.
Maintenance	After deployment: fixing bugs, updating drug databases, adding features (e.g., e-prescriptions) and adapting to new regulations.	Keeps the software useful, secure and up to date throughout its life.

(c) Reduction of Risks and Improvement of Quality

SDLC provides a structured, phase-wise framework with defined deliverables and review points. Feasibility study and requirement analysis uncover technical, cost and regulatory risks early, when they are cheapest to fix. Design reviews prevent architectural errors, and systematic testing catches defects before they reach the pharmacy, where an error could affect patient safety. Documentation, planning and progress tracking improve control over time and budget, while the maintenance phase ensures continuous improvement. As a result, the project delivers a reliable, high-quality product that meets user needs.

4. Smart E-Commerce System – SDLC models (5 Marks)

(a) SDLC Models

1. **Waterfall Model:** A linear, sequential model in which each phase (requirements, design, implementation, testing, deployment, maintenance) is completed before the next begins, with no going back. It suits stable, well-defined requirements.
2. **V-Model:** An extension of Waterfall in which every development phase has a corresponding testing phase (requirements ↔ acceptance testing, design ↔ integration testing, coding ↔ unit testing). It emphasises verification and validation, but is rigid.
3. **Incremental Model:** The system is built and delivered in small functional parts (increments). Each increment goes through analysis, design, coding and testing, and the product grows with every release, e.g., first catalogue and cart, then payments, then recommendations.
4. **Agile Model:** An iterative, collaborative approach that delivers working software in short cycles (sprints of 1–4 weeks) with continuous customer feedback, welcoming changing requirements. Examples: Scrum and Kanban.

(b) Comparison of Waterfall and Incremental Models

Basis	Waterfall Model	Incremental Model
Approach	Single-pass, sequential	Multiple small iterations of increments
Delivery	Complete product at the end	Working parts delivered early and repeatedly
Requirement changes	Difficult and costly	Easier to accommodate in later increments
Customer feedback	Only at the end	After every increment
Risk	High; problems found late	Lower; risks are found early
Testing	After implementation	Performed for each increment
Suitability	Small projects with fixed requirements	Large projects where requirements evolve

(c) Recommended Model: Agile

Agile is the most suitable model for the Smart E-Commerce System. Customer requirements change frequently because of market demands, and Agile handles change through short sprints and backlog reprioritisation. Working features (search, cart, checkout, payment) are released early, which gives faster time-to-market and a competitive advantage. Continuous feedback from customers and stakeholders keeps the product aligned with user expectations, while regular testing and retrospectives maintain quality and reduce risk. Waterfall and V-Model are too rigid for such a dynamic environment, and Incremental lacks Agile's strong emphasis on continuous feedback and adaptability.

5. Smart Hospital Information System – UML (5 Marks)

(a) Role of UML in Object-Oriented Software Design

The Unified Modeling Language (UML) is a standard, graphical language for visualising, specifying, constructing and documenting the artefacts of a software system. In object-oriented design it provides a common notation for describing the structure (classes, objects, relationships) and behaviour (interactions, workflows) of the system, helping analysts, designers, developers and clients communicate clearly before any code is written.

(b) UML Diagrams

- **Use Case Diagram:** Shows the functional requirements from the user's viewpoint: actors (Patient, Doctor, Receptionist, Pharmacist) and use cases (Register Patient, Book Appointment, Prescribe Medicine, Generate Bill) with their relationships such as include and extend.
- **Sequence Diagram:** An interaction diagram that shows how objects exchange messages in time order, using lifelines and arrows. Example: Patient → Reception UI → AppointmentService → Database for booking an appointment.
- **Communication Diagram:** Shows the same interactions as a sequence diagram but emphasises the structural links between objects, with numbered messages (1: requestAppointment(), 1.1: checkAvailability()), highlighting who communicates with whom.
- **Activity Diagram:** A flowchart-like diagram that models the workflow or business process using activities, decisions, forks/joins and swimlanes. Example: patient admission → registration → doctor consultation → tests → billing → discharge.

(c) Support for Analysis, Design and Documentation

In analysis, use case and activity diagrams capture and validate requirements with hospital staff. In design, sequence and communication diagrams define object collaboration and responsibilities, which guide coding and database design. For documentation, the diagrams form a standard visual record that new team members can understand quickly and that eases maintenance and future enhancement. They also help detect errors and omissions early, reducing cost and rework.

6. Smart Railway Reservation System – object-oriented design (5 Marks)

(a) Concept of Object-Oriented Software Design

Object-oriented design (OOD) is a methodology in which a software system is modelled as a collection of interacting objects, each combining data and behaviour. The problem domain is analysed to identify classes such as Passenger, Train, Ticket, Seat, Payment and Schedule, along with their attributes, methods and relationships. OOD is based on abstraction, encapsulation, inheritance and polymorphism, and focuses on building modular, reusable and maintainable software.

(b) Structured vs Object-Oriented Approach

Basis	Structured Approach	Object-Oriented Approach
Focus	Functions and procedures	Objects and classes
Decomposition	Top-down, divides into functions	Bottom-up, identifies objects first
Data handling	Data is global and separate from functions	Data and functions are bound together
Data security	Low; data is open to all functions	High; encapsulation and access control
Reusability	Limited	High, through inheritance and composition
Modelling tools	DFDs, flowcharts, ER diagrams	UML diagrams
Maintenance	Difficult for large systems	Easier; changes are localised
Example	Separate functions bookTicket(), cancelTicket()	Ticket object with book() and cancel() methods

(c) Improvement in Flexibility, Scalability and Reusability

- **Flexibility:** Polymorphism and encapsulation allow changes without affecting other parts. For example, a new payment type is added by creating a new class that implements the Payment interface.
- **Scalability:** The modular object structure lets the system grow easily: new trains, classes, routes or services (e.g., Tatkal booking) are added as new objects or extensions as the number of passengers increases.
- **Reusability:** Inheritance and composition let existing classes be reused: AC, Sleeper and General coaches inherit from a common Coach class, and classes such as Payment or User can be reused in other transport systems.

Thus, OOD produces a railway reservation system that is easier to extend, test and maintain over time.

RUBRICS FOR CALCULATION

SHORT ANSWER TEST

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Understanding of Concepts	25%	Demonstrates thorough, accurate understanding of concepts	Good understanding with minor gaps	Basic understanding with some misconceptions	Poor understanding; major misconceptions
Explanation	25%	Detailed, well-explained answers with relevant examples	Clear explanation with adequate details	Limited explanation; lacks depth	Poor or unclear explanation
Application	20%	Effectively applies concepts with strong analytical ability	Applies concepts with minor errors	Limited application with weak analysis	No application or incorrect analysis
Organization	20%	Well-structured answers with logical flow and coherence	Organized with minor issues in flow	Basic structure, lacks clarity	Poor organization, incoherent answers
Accuracy	10%	Completely accurate and covers all required points	Mostly accurate with minor omissions	Partially correct with missing points	Incorrect answers with major gaps